

C 语言程序设计

第七讲 函数与模块化 递归算法与递归函数



张 华

递归算法与递归函数

- 递归算法
- 递归函数
- 程序设计举例

案例分析：递归算法

用递归方法计算n!。

分析

➤ $5! = 5 * 4 * 3 * 2 * 1$

➤ $5! = 5 * 4!$

➤ $4! = 4 * 3!$

➤ ...

➤ $1! = 1$

递归公式

$$n! = \begin{cases} 1 & (n=0 \text{ 或 } 1) \\ n * (n-1)! & (n > 1) \end{cases}$$

用迭代法实现

```
s=1;
n=1;
while (n<=5)
{ s = n * s;    n++; }
```

$$n! = n * (n-1)!$$

基本情形

简化问题

案例分析：递归算法

举例：用递归方法计算n!。

源代码 (cw0805.c)

```
#include <stdio.h>

long fact(int n) {
    long f; //保存n!
    if (n==0||n==1)
        f=1;
    else
        f=n*fact(n-1); //递归调用

    printf("\t%d!=%ld\n", n, f);
    return f;
}

void main( ) {
    printf("\n\t5!=%ld\n", fact(5));
}
```

1!=1
2!=2
3!=6
4!=24
5!=120

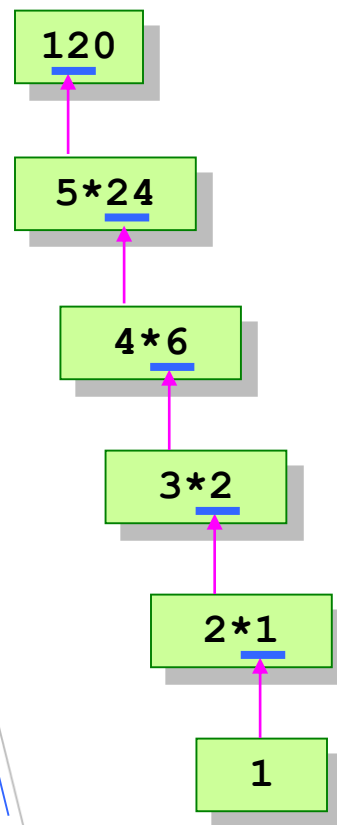
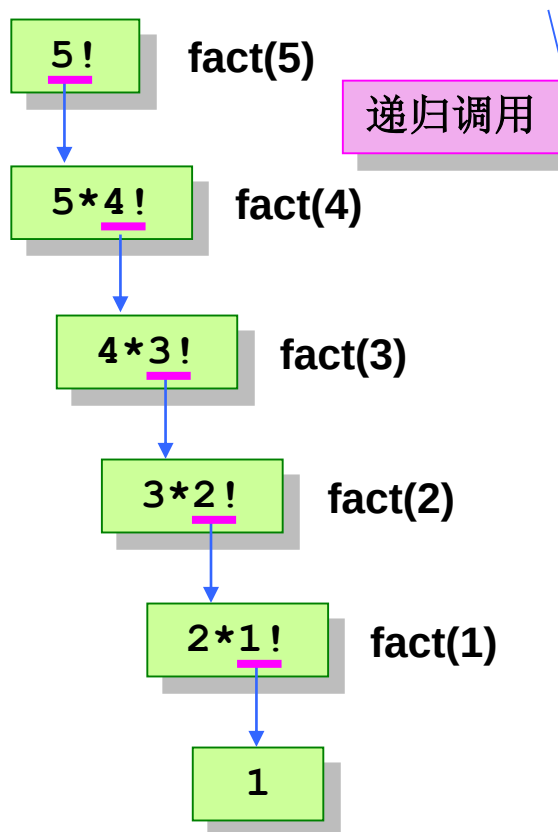
5!=120

递归调用

案例分析：递归算法

举例：用递归方法计算 $n!$ 。

分析

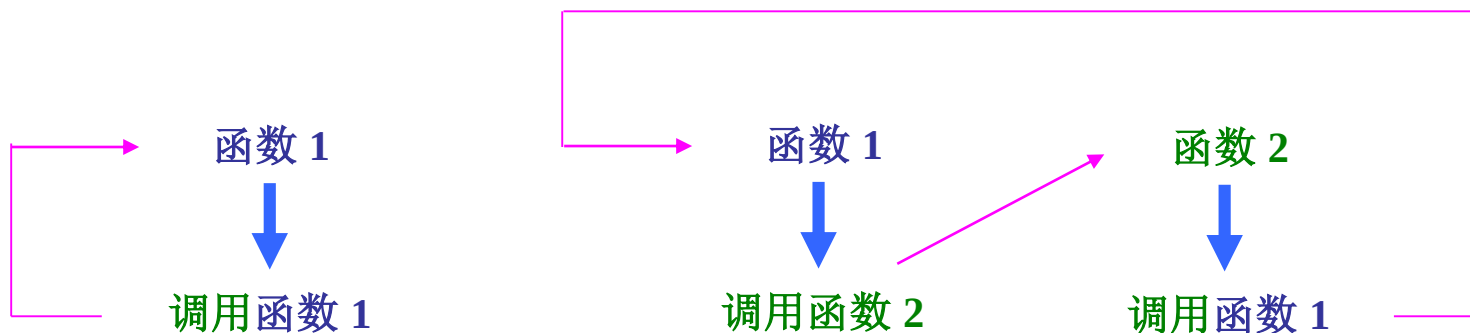


从递归调用返回值

递归函数

递归函数

✿ 直接或间接调用自己的函数。



用递归方法解决问题小结

用递归（函数）方法解决问题

✿ 递归函数只知道如何去解最简单的情形（基本情形）

➤ 例如，简单的返回一个值

✿ 把复杂的问题分成两部分：

➤ 可以直接求解的部分

➤ 不能直接求解的部分

– 这部分的问题与原问题相似，且复杂度降低

– 函数可以通过调用自己来解决这个问题（递归调用）

✿ 最终遇到基本情形

➤ 函数识别出基本情形，将结果返回给前一个情形

➤ 一系列的结果按顺序返回

➤ 直到把最终结果返回给原始的调用者（可能是main()函数）

递归算法练习

使用递归方法计算斐波拉契（Fibonacci）数列。

1 1 2 3 5 8 ...

✿ 分析

- 第一、二个数是确定的。
- 从第三个数开始，每个数是前两个数的和。

✿ 递归公式

- $\text{fibo}(1) = 1$
- $\text{fibo}(2) = 1$
- $\text{fibo}(n) = \text{fibo}(n-1) + \text{fibo}(n-2)$

递归与迭代

递归与迭代的比较

✿ 循环

- 迭代：明确使用了循环结构
- 递归：重复调用递归函数

✿ 终止条件

- 迭代：循环条件不满足
- 递归：遇到基本情形

✿ 都有可能出现无限循环

✿ 如何选择

- 迭代：性能好
- 递归：可读性好

程序设计举例：汉诺塔问题

汉诺塔问题

问题

- 假设有三个分别命名为X，Y和Z的塔座，在塔座X上按从小到大的顺序放了n个大小不相同的圆盘，并依次编号为1，2，...，n。现在，要求将X上的n个圆盘移到塔座Z上，并按同样的顺序叠放。
- 移动时必须遵循以下规则：
 - 每次只能移动一个圆盘；
 - 圆盘可以插在X，Y和Z中的任一塔座上；
 - 任何时候都不能将一个较大的圆盘放在较小的圆盘上面。



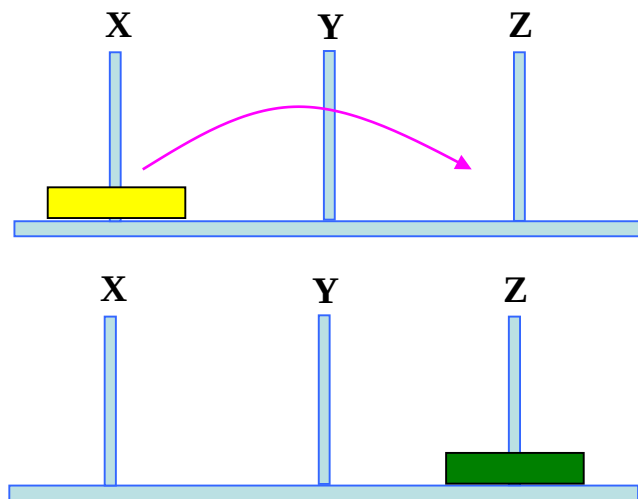
程序设计举例：汉诺塔问题

汉诺塔问题

分析

➤ $n=1$ 时

- 将圆盘1从塔座X移到塔座Z。



基本情形

程序设计举例：汉诺塔问题

汉诺塔问题

分析

➤ $n > 1$ 时

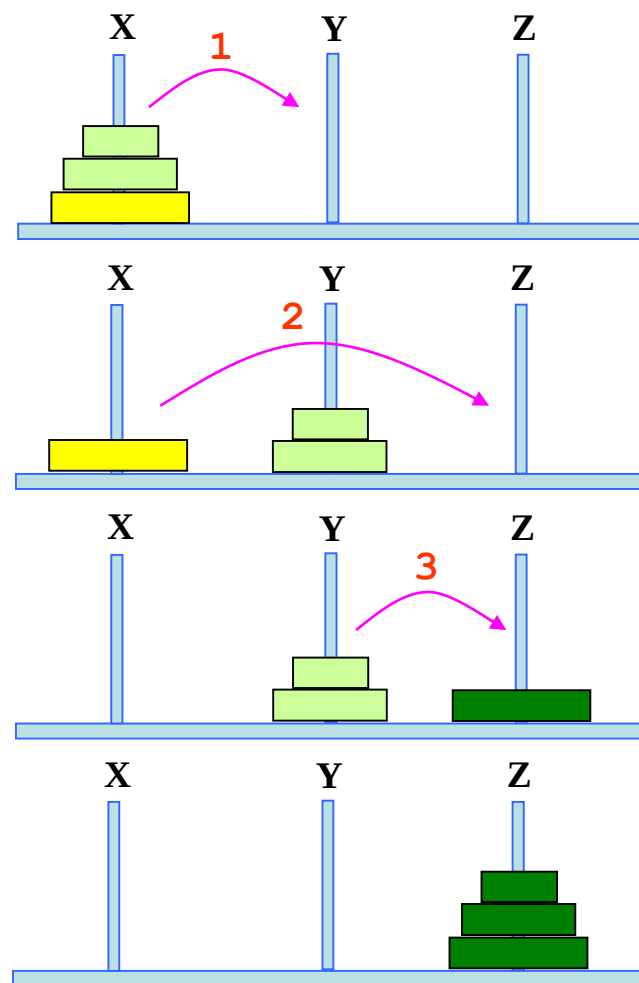
1. 利用塔座z为辅助塔座，将压在圆盘n之上的 $n-1$ 个盘从塔座x移到塔座y;

(—与原问题类似)

2. 将圆盘n从塔座x移到塔座z;

3. 利用塔座x为辅助塔座，将塔座y上的 $n-1$ 个圆盘移动到塔座z。

(—与原问题类似)



程序设计举例：汉诺塔问题

汉诺塔问题

设计

➤ **move** 函数：移动一个盘

- 把编号为 **n** 的盘 从 **s** 塔移到 **d** 塔

➤ **hanoi** 函数：移动 **n** 个盘的汉诺塔问题

- 把 **n** 个盘从 **x** 塔移到 **z** 塔，**y** 塔作为辅助塔

```
void move(int n, char s, char d);  
void hanoi(int n, char x, char y, char z);
```

程序设计举例：汉诺塔问题

汉诺塔问题

✿ 实现 (cw0806.c)

```
#include <stdio.h>

void move(int n, char s, char d);
void hanoi(int n, char x, char y, char z);

void main() {
    int n;

    printf("\t Input the number of disks:");
    scanf("%d", &n);

    hanoi(n, 'X', 'Y', 'Z');
}
```

程序设计举例：汉诺塔问题

汉诺塔问题

实现

```
void hanoi(int n, char x, char y, char z) {  
    if (n==1) move(n, x, z);  
    else {  
        hanoi(n-1, x, z, y);  
        move(n, x, z);  
        hanoi(n-1, y, x, z);  
    }  
}  
  
void move(int n, char s, char d) {  
    printf("\t%d\t%c-->%c\n", n, s, d);  
}
```

```
Input the number of disks:3  
1      X-->Z  
2      X-->Y  
1      Z-->Y  
3      X-->Z  
1      Y-->X  
2      Y-->Z  
1      X-->Z
```

小结

- 函数可以调用自身，这种调用方式称为递归。
- 有些问题适合采用递归方法，但是递归会在内存使用和时间花费方面效率低下。